

Hello, good morning, good afternoon, good evening folks, depending from where you guys are joining in. How is everyone doing today? Let me know in the chat. Yes, I know you guys could hear music and no voice and sorry I was lip-syncing. It's not the first time you're hearing to these songs. I have played these songs a lot of times in different sessions. Even though they are AI-generated songs, I've heard it enough that I now remember those songs because of which I start singing along. So sorry for that. It's 1 a.m. in Sydney, damn. It's gonna be a long night for you, Prasad, but it's gonna be worth it. Can be a couple of more cameras on as well, folks. Only my good friend Krishna Kishore is here. Rest, I feel, is not here. Alright, good. Folks, it'd be really helpful if even the mentor can see you folks when he or she is teaching so that, you know, they can also understand if you guys are understanding. It's okay, some of you guys, if you cannot switch on, but if some of you guys can while the teaching also is going on, it also helps the mentor to relate with you better. And for someone who's asking, how are people adding Fireflies? It's basically directly connected with the email ID that they are joining in, they are meeting, they block their calendar, so it automatically joins. By chance it's not joining automatically, you can go on Fireflies and you can click on capture and you can upload it. You don't need Fireflies, but if someone wants it, you can add it as well, that's your call. Alright, everyone is doing well, I hope. Alright so folks before we start a couple of announcements very quickly today is the day one today finally folks you starting with the engineering accelerator so when I give you a very very warm welcome for the engineering accelerator and folks this is where our journey starts together for the next 14-15 days you are going to hear my voice you're gonna see me continuously because I'm gonna come and bother in between as well don't worry okay folks i am not taking any thing regarding tools or anything over here for tools or any questions i have already mentioned there is something called as uh you can go on troubleshooting you can go on the circle if you want i'll quickly show you once again just go on circle go on troubleshooting and you can block a call there where the team will get in touch with you personally and help you with the tools per se so if you want i'll show you once again yeah This is the page that you will see. Welcome and next steps. Over here you see book your troubleshoot call. Just book your troubleshoot call. The team will help you out. Alright? Very simple. Where is the updated schedule? Good question, Ravi. That's a good question. Don't worry, the same schedule is going to be updated. I'm sorry, due to a little health issues, I was not able to do it yesterday. I'm still at home. As you can see, I'm not in office. I was a little unwell, but I thought that I'll come. Rather, you don't have missed anything. The first session that we've done is only in terms of Python. We did Python Basecamp where we went through environment setup and all those different aspects of Python so that later on when the session is going on, you won't have questions regarding Python. Don't worry, regarding the schedule, I'm going to do it right now. As soon as your session would start, I'll go behind, I'll get it done, I'll drop a text that it's done so that you can go and check it as well. That being said, like I was mentioning, this is day one of your engineering accelerator. we're gonna start slow folks uh we're gonna start with prompting chat completion what chat completion is you might some people would know some people wouldn't that's fine we'll cover what it is and how important it is now for prompting we will also do something that i love we not be generally called xml prompting and json prompting What XML system prompting is, that Siddharth will cover. For people who don't know King Siddharth, he was the one who took the onboarding call on Friday for you folks. He is someone who is a head of product and design as well. Proper, you could say, someone who's from an AI background, who loves to talk and teach about AI at OutSkill itself. That's about it. Now, before I start with the sessions, I want to tell you where I will be sharing the codes. This is also something that I will build once the session starts, okay? Give me one second. I'm just setting things up. So I'm going to show you a previous batch for now. I'm also updating your current batch. So every day, we will cover a couple of codes. So today might be a little less code, but going ahead, we'll do a lot of codes as well. So I will create a GitHub repo. I hope you guys know what a GitHub repo is. Folks, do you guys know what a GitHub repo is?

Yes, no, maybe in the chat. Do you know what GitHub is? Okay, I got one no. It's okay if you don't know. Don't feel the pressure. It's okay. You can say no if you don't. That's fine. So folks, this is a platform where we generally upload any kinds of codes. You want to store your codes and all those. That's where GitHub is used in simplest example that I can give. Okay. So this is what your GitHub would look like. Now I will change it every day before the session. I'll upload a code. For example, tomorrow's J2 sessions code, I will upload tomorrow afternoon, where you can go and you can check these kinds of codes also that will be available. Sometimes we'll provide to you both a file which is there on Colab and the code as well in a .py file. So all of these codes will be available to you folks over here. This is one GitHub repo that you will get access to, but only view access, okay? Clear so far? If I'm going a little fast, you can let me know. I will share the URL also. I'll share the URL, don't worry. First, I just need to clear a few aspects. Okay, repeat. We will create a GitHub repo where I will upload all the codes that is taught to you in the next 14-15 days. And every day, we'll upload it in the morning so that you can go through it if you want. Some of the codes will be heavy and I don't want you to get overwhelmed. It is fine. We will take you through the code. We'll also explain you what the concept behind it is, okay? That is the first repo. Now, the second repo. Stop sharing for that. One second. It's something that we call as a submission. So during this course of time, we will also do a lot of breakout rooms and assignments that I will personally give. But how to do this is basically, one second, share, yeah. So there'll be a similar repo like this where I will create folders with your name. For example, I can see a name like Krishna Kishore. So it will have like Krishna underscore Kishore. Second name if I see as... Sushma Acharya. If I'm pronouncing any names wrong, I'm very sorry. I'm just giving a heads up. It will be like Sushma underscore Acharya. So all your names would be over here, where you'll open and you'll be submitting whatever assignments we will provide. Okay? Now, but how will you submit? You will not have access to these repos. So for that, my team, when I say my team, it's generally Shyam. Shyam will share a simple form with you. What you have to do on that form is put your email id now which email id you will put over there is the email id that you log in on circle which basically the email id that you paid okay email id name phone number and github credential now what github credential is i'm going to show you give me one second because this has been an issue previously so i don't want any issues happening second just going to log in okay so folks how you can share is First of all, go and create your GitHub accounts. It is very simple to create a GitHub account. Just go on github.com and log in with Google if you don't have already. If you have, that is great. This is the home page that you will come to. Now, what you need to put in that form is, not join the WhatsApp community. I would advise join now. Okay, so what we will do is, Shyam, can we also share it on LMS, on Circle? Yeah, I'll be uploading on LMS as well as Circle. Alright. Yeah, if you already have a GitHub account, you can use the same. You don't need to create a new one. We will also share it on LMS and we will drop the link where we have uploaded it so that you can go and do it. Does that help? No, I cannot. uh rather not ms lms that is circle basically where all your recordings and all resources are available this platform.outscale.com this is your lms you can log in here and you will have all the access live session resources recordings introductions i can see barely 15 introductions by the way folks please introduce yourselves i don't know who you guys are how will i connect with you folks please put your introductions over here okay any questions on this No, no, no, no, no. You don't need to have the same email ID. You can use any email ID that you already have. If you already have an account on GitHub and it's a different email ID compared to which is there on LMS, that is fine. Your GitHub email ID does not matter. Only your GitHub credential matters to me. Your username on GitHub, basically, in easiest words. Okay? All right, folks, are we ready to start? Are you guys excited? If yes, for any questions that you may have regarding GitHub, I'll take it on the chat. But are you guys ready to start with your day one? I want to see a little energy in the chat. All right. Awesome. So without wasting any further ado, Siddharth is already on the call. How's it going, folks? How's it going? Go for it, Karthik. Sorry to

interrupt. No worries. Folks, can we give a very warm welcome to Siddharth in the chat? Can we welcome Sid in the chat? All right. Super excited to get you folks started. Let's talk to some AI. I should get my face out of your faces. Just a couple of minutes. I'll just share my screen. As usual, we'll do a whiteboard and I will share it at the end of the session. We will share it afterwards on LMS as well. So don't worry, this will be available to you. For now, we'll pay attention to what we are up to. All right. So let's get started. And we have a lot to cover. But before I even get there, you know, I found it very, I find it very almost like intellectually offensive if somebody just jumps into, hey, this is, you know, what happens without telling me the why behind it. So if you were, you know, you folks are getting started with AI, what do you think is the first logical step is as an AI native engineer? What is the first thing you should know how to do with AI? Learn prompting, sure, but where will you run the prompt? Where will you run the prompt? Fundamentals, what are the fundamentals? Let's articulate. Why AI? Well, that's a question you might have asked earlier. LLM, command line from code, okay, LLMs, tools, all right. Let's say I have an AI model, all right. Let's draw an AI model and let's, this is a large language model, all right. If I gave you, I sent you a file, all right, and I told you this is an LLM, it does great coding. What is the next thing you will have to do here? What is the next thing you will have to do here? To even send a prompt. Automate my work. Automate comes afterwards. UI to LLM, transformer. We want to be able to use it. And run host correct. Pavitra is the first one to get correct. We want to run and host it. And basically, we want to send it prompts. And it will respond to us. And this is also called completions. Or you can look at them as inputs and outputs. I'll send input and it'll give me an output, which is what most programs do. Your xKali are not accessible. I don't know, Hemabindu, what do you mean? Shyam already mentioned that, folks, you will have to log in over there with the email ID. Just log in and then you'll get an option to download. The Dropbox, yes, you'll have to log in. We have scored free extension and it'll open directly. Dropbox isn't, I'll check, I'll check later. For now, we'll focus on this. Now, inputs and outputs have to be, you know, then this is what we can call, you know, in common language, running an LLM, right? Or we'll call, another technical word you will come across is inference. It's also called test time. right why it's called lexicry is not that important but this you are doing inference on the AI model uh you're doing test time closely all these words are used exchange interchangeably but we are running this all right now majority of us will not run it and honestly running it is not that big a deal um in fact if you want to run a local you know llm we'll talk about that during um Hugging face, discussions, etc. And more than OLama, I like LM Studio. I'm just going to put that in a grey bracket for you to look into later. The reason I like LM Studio over OLama, because they show you a nice GUI and tell you in green or red or orange, can your system handle it or not. So LLM Studio is my recommended way of running AI. And in servers, etc., there are a whole bunch of stuff that you can use. Again, those are most likely at scale, you're not going to be running the inference, etc., because it economically does not make sense to compete with OpenAI and Anthropic on GPU pricing. It just doesn't make sense. Anywhere you look at a provider, it's going to be much cheaper. So we're not going to run it, but we are still going to want this input-output interface, right? We still want to be able to connect with it and, you know, get these inputs out of this. And the crazier part still, there are multiple different providers and different models. Can anybody tell me all the labs, all the labs that are there? First one is OpenAI. Which other labs make AI models that you can use? Any other labs at Anthropic? Yes. Which else? Can you imagine this will be an integration nightmare like everybody has their own favorites and everybody has them you know thing going on you know how tech works But here, the fascinating thing, tech worked differently. Sarvam is also there, sure. Let me put in that name. So you have a whole bunch of these providers. And you have Bedrock, etc., which can provide you proprietary models and open models. Now, wouldn't it be crazy if there was one common way, one powerful, sorted way to do this input-output inference? Using almost any model, practically any model. What if there was one common API? Wouldn't that be nice? Wouldn't that be nice? Yes. And in

fact, that API does exist. Not OpenRouter. OpenRouter is an aggregator. Though it makes that entire thing much more easy. There's a pattern. There's an API pattern that was created by OpenAI. And it is called ChatCompletions API. Chat completion API. Very old API. Many people feel it's outdated. But yeah, it's the only common one supported. Otherwise, everyone has their own thing going on, and everybody has their own preference. Some API designs are terrible, like Rock API. So it's all over the place. So chat completion API is basically a very, very old standard. And what is the interface? I'll come to that. And chat completion essentially works Mostly anywhere. In fact, it is also known as OpenAI standard or OpenAI SDK standard. And please remember, chat completion and completion API are not the same. Chat completion is a very specific interface. All right. And we will talk about this today because this can unlock for you. Any practical, you know, LLM on the planet. And if you have code written, you know, in chat completion sort of way, can you remove AI glasses? Oh, I'm so sorry. I completely forgot. Thanks for reminding me that. I completely forgot that I had that effect going on. It should be so non-serious for me to like, you know, continue that. Okay. Remove that. There you go. Now I'm a human again. All right. Thanks a lot for mentioning that. Now, this is what it's called. And... It is supported practically by anyone. In fact, if you enter today's AI market, you'd be kind of dumb not to support chat completion API because most people, you know, have integrations, you know, already built here and it goes, it becomes crazy. In fact, you know, you know what? This is the first time I'm seeing this in tech. OpenAI built their SDK first. They were the first lab to build an SDK. So, not only people, like all the labs that came after them said, we will support, like if you have code written in ChatCompletion API, we will support that as it is. In fact, they said, we will even support the OpenAI SDK itself. You can actually use OpenAI SDK to connect to Anthropic, to Mistral, to NVIDIA, to XAI and not kidding. Okay, because OpenAI was the first one to get there, so a lot of people wrote integrations and I think this is a really smart move that benefit tech in this way because I've never seen this kind of coming together happen. And you'll be surprised that you can actually use OpenAI SDK to, you know, call other people's API, all right? It's fairly easy to do. So now let's look at chat completion API. But before we get into chat completion, I want to, you know, talk about a particular shape of data because this chat thing, you know, looks a particular way. So let's go to ChatGP right here, and I will go to instant to get faster answers. What is the capital of France? All right. Then tell me 10 fascinating facts about Paris. All right. Can you step back and summarize and go forward, please? It's a bit fast. Sure, I will. I'll come back and circle back again, but let's just quickly take a look at what's happening. I'm just prompting. I'm just having a conversation with AI. I'm just chatting with AI. And what I'm doing here is, folks, exactly what I spoke about. I'm sending it prompts, and it's replying to me. It's sending me responses. I am sending it inputs, and it's giving me outputs. And this is what we call a multi-turned chat or multi-turned chat. Let me draw this for you. And we'll talk about how that's related. So the shape looks like something like this. You have, you know, first message coming in from me. Let's draw it like this. Which is, what is the capital of France? Let's call it the prompt. And AI responded, the capital of France is Paris. Then I said, take the top 10 fascinating facts about Paris. And then it responded with a bunch of facts. So everyone knows what this like when I draw the shape what I mean by the shape of the data there are alternative messages yes no this is how we chat all right this is called multi-turn chat all right now what is multi-turn chat I think chat is pretty obvious that you're chatting this is exactly what chat looks like and multi-turn is that you know there are turns so I start an AI response that is typically one turn right so this is turn number one right and there is turn number two so those are the turns and this is what is meant by multiple terms are possible it's not just one term and what we call these inside are messages this is this is a message this is also a message but this message is from AI all right i'm just breaking down the shape for you folks so you know it becomes a little more familiar these are just messages in the chat very similar to you know how we would look at things they're differentiated because this one is by user this one is by AI and

traditionally this is how we do that we give it a role and this one goes with role user and this one usually is with role assistant all right so this is how we identify them in a you know you know sort of a database and you can store messages and that makes your entire chat your conversation on your site right this entire thing can be called or like we can call it chat a conversation thread all these things are possible so this is what we call a multi-turn chat another way to look at this entire piece what's happening behind the scenes you're sending in prompt there's an llm running and then it starts doing completion it starts sending outputs right and this is how it manifests in front purpose and these are like the same process described two different ways here prompts and completions are shown as a process data flow here i'm showing you the shape of the final data Everyone clear of how this running in LLM and how multiturn chat are related? Any questions? Any questions so far, folks? Clear? Awesome. So, we take a bunch of messages, right? This is nothing but a bunch of, a list of messages. Or rather, let's say, ordered list of messages. Right? So, we can represent a list in Python very easily. And we'll just call it, like, you know, I'll write it something like this. Messages will be a list. This one will have a role and let's call it user. And this will have content saying what is the capital of France. This is basically turning it into an API. And there will be another message. I'll have multiple such messages in my list. There's an array for those people. We call it list in Python. They are slightly different from arrays. But you can imagine the next turn will be, you know, assistant. And it will say the capital of France is Paris or it can just say Paris, right? And so on and so forth. Like you can just have the list go on. And this is the first part of the entire deal. We have messages going on. There will be role tool. Yes, there are role tools. You will learn that about when we talk about agents. Are there other shapes of data? Yes. But today I will not get into those because then they can become potentially confusing. If we get some extra time later on, I'm happy to dive deeper into it. JSON with key value pair is called one-turn. I'm representing the one-turn concept in JSON values. It's not necessarily, it doesn't have to be JSON. right the truth is this is a chat format you can put it in json xml it doesn't matter right so we have taken messages this is the first part and who are we talking to folks who are we talking to here who is the assistant here who is the assistant here is there multiple users not in the simple one llm and llm is typically an ai model right and do models have identities can you identify individual models tell me an example of a model Jamil bhai, aapne ye jo share kiya hai screen, waha par Jamil bhai ke naam ke upar audio mute dikha rahe hai. Unmute kar ke dekhiye, ho sakta hai voice aaj hai. Aapki aawaz aari hai, ye jo mujhe screen dikha rahe hai na, uske upar jo Jamil hai, uska unmute hai. There are a whole bunch of models. Those of you not sure, let me quickly show you. On the app, you talk to an AI model, and this is where you can see the models. There are two Gemmys, correct? Claude is the app, and these are the models. Similarly, Gemini has models as well. For example, the lab, the company is Anthropic. The app is Claude.ai. The model is Sonnet 4.6. Similarly, here, OpenAI is the lab, the company. They have an app called ChatGPT. And they have their own models. This continues even if you go to somebody else's, you know, people who don't have their own model, like Perplexity. Perplexity, the company and app name is same, Perplexity. And they offer a whole bunch of models. But none of them like it. It's not working. So they have an offering as well, but it's not that impressive compared to Gemini or Prod. One second, folks. Right, yeah. So... That is the difference between apps and models. And what we need, the two basic things you need to talk to an AI are actually just these two properties. Which model do you want to talk to? And what do you want to talk about? And if you share this shape, this shape, this interface of model and messages, these two parameters, if you club them together, This is exactly what is the shape of this thing called chat completions API. This is the chat completion interface. That's all you need to basically invoke any AI on the planet. There are other ways, there are better ways, there are different ways, there are more options, but this is the fundamental basis. Which model and what is the history of conversation? So, you know, a lot of people believe AI has memory. LLMs don't have memory. The memory feature they give you is very different. But if I go here, what,

you know, were we talking about? Do you think it will be able to answer that we were talking about Paris? Do you believe it will be able to answer? If I ask it this question? No? Okay. Yes? Okay. Let me put that. Meena, yes, you can. You can use different model for each term. There you go, it remembers. So, if AI models don't have memory, how does it remember what's happening? How does it know? STM, context pass, session memory, correct. So, we are the ones storing the entire chat conversation, or OpenAI can do it for you as well, and we send it back to LLM. We send the entire chat back and then get the response. That's how it remembers. So all of this data is stored in database and then sent back to AI. AI personally does not remember. So that is very, very important piece to remember. And that is why this shape looks like this. And let me actually show you what this looks like in SDK, et cetera. The code is very simple. In fact, we can ask AI to implement it right here. Let me actually ask that to focus. Can you build me a basic LLM inference using OpenRouter API key? Let's ask it to build this for us. Meanwhile, while it builds it, I want to quickly show you what these documentation... So all labs have their API documentation. OpenAI API docs. I will open Anthropic API docs, and I will open Gemini API docs. These are all API documentations. And first and foremost, I want to show you the chat completion API. How does multi-LLM app does it with multiple modules? I'll talk about that. Where are we storing the data in browser? I mean, it's stored in the cache, the browser cache, through stored shape, yes. So, okay, this is OpenAPI, not OpenAI. There you go. So developers.openai.com forward slash docs. Here you will see a different thing. You will see a response API. This is the new API they are trying to promote. However, chat completion. Assalamu alaikum, Sati. Yasin bhai, Faiza, and Mohamed Omar, and anybody else who has joined the call. We are not able to fix the audio problem. Once again, I will show you probably in Python. Since Python is our go-to language, we install and import OpenAI from OpenAI package, initialize the client, and then we get a completion by calling literally `client.chat.completion.create`. You don't have to remember this. You literally start typing, you import this and the next line, it will just use this. As you can see here, we set the model and we send the message history. And then we get a response back, which looks something like this. And in the response, we will have another message with role assistant. And the content is right here. Hello, how can I assist you today? This is what it sends back. From here, we can get that. And essentially, that's our basic chat completion. This is create chat completion. There is list chat completion, et cetera, as well. You can stream it as well. And this is on OpenAI docs. There are other things that are supported. For example, apart from messages and model, you can also send audio directly. You can set something called frequency penalty. There are a lot of inference things that are available. All right. And you can set max completion tokens as well. We'll talk about this during our session. All right. And you can also have various Like options, for example, you can ask it to specifically respond in JSON schemas or JSON objects. We'll talk about this in structured responses. You can set a seed. There's a whole bunch of stuff that is out there for you, right? But model and messages is the heart and soul of all of this. And now I'll show you the same thing on Claude platform. somebody's asking can you describe inference for now i will say running an llm so i when i ask something it gives me a response when i send a prompt it gives me a completion when i send an input it gives me an output running that llm is what we're calling inference there are deeper meaning this as well but i think this one is good enough for us start with now i am on the developer guide i am going to api reference all right and here i will search for chat completion all right i will search for openai instead OpenAI SDK compatibility. So this is what they call it. Everybody calls it slightly differently. Let me add that term here. OpenAI SDK compatibility. This is Anthropic. It runs Claude. All right. And they are compatible with OpenAI, you know, AI start completion. However, they do have certain suggestions. You know, it can affect quality, etc. But this is what this looks like. Again, OpenAI chat completion create, roll messages, design the same thing, and you can set their model. And they have additional optional stuff like thinking token, etc. Literally the same API. In fact, they're not even using their own SDK, like I said. And finally, Google. They

don't even acknowledge they have competitors, let alone put a name and link of their competitor in their documentation. And I want you to behold this rare happening in the world of, you know, whichever world we live in. Here I can go to right here guides and OpenAI compatibility. And there you go. Here we see Python. You literally import OpenAI SDK. We set the Gemini API key and change the base URL so it doesn't go to OpenAI. It goes to Google APIs. We set the model. And we get the messages. And this is how you talk to an AI. And all these are additional options that are available, that are Gemini-specific. Okay? Now, so that was a quick way to show you that it's practically available everywhere. However, unless you're going batch inference, which we'll talk about during pause session, you might want to use a unified interface for LLMs. I really have OpenRouter for this. And if I go to the documentation, Again, the primary way you use them is chat completion API. If I go here, quick start, they have changed it to chats.send, but you can see the same shape still applies. It's essentially, you know, OpenAI chat completion format. So is it good? What is SDK compatible means? It means that you can just use their SDK. You don't have to install another software. Are these stateless APIs? Yes, as far as I am aware, they are stateless APIs. And do we need to integrate with these APIs while building an agent or is it taken care of by LLM itself? If you're, I mean, you will, you have both options. You can do automations in UI with something like madeNATN or something like that. But if you're writing code, yes, either you or the framework has to take care of this. What is the contract for the send method? That, If you want to, like, do inference over HTTPS, I can show you that. This is the default one. Let's change it to HTTP. And this is, essentially, you have to send your, you know, authorization bearer token, JWT style, and that's it. There's no other special header. I think content type is the only other header that you send. And the rest is literally the same. That's a contract. With QA framework, these integrations are out of the box. Not all of them. What is the contract percent already covered? Does the API call work the same if the model is deployed locally? Yes, it works the same even if model is deployed locally. For example, I'll try to open LM Studio for myself. One second. And I can show you this ChatCompletion API running on my local server. But everyone is clear that everyone supports ChatCompletion API. Everyone clear about that? That this is the universal way to integrate with AI models. Are other parameters completion API across providers? Yes, there are many other parameters. They are different for different providers. This is where it becomes an unholy mess. There is, for example, there's something called max tokens or max output tokens. Very confusing. Which one to use where? So like the typical tech thing happens, everybody has their own API of the same underlying thing. But at least for the basics, it makes sense. All right, I can show you a couple of things which are unique to each model. For example, let's take a look at how Gemini handles when we send an image. Let's say we send it an image and we wanted to answer some questions based on it. You can see they send an extra body. An extra body is literally just JSON again defined. And here we are setting thinking level low. Thinking is one. And let me show you images. There you go, image understanding. So almost all of them will support base 64 encoded file input, but check this out. So this is a message, a single message role user, and our content is now a list, not a single string. This is text, what is this image? And this is image URL. The URL itself is the base encoded 64 image. And this is how Gemini supports images. Now let's see how OpenAI does it. All right. Let's take a quick look at it. They have not given a thing here, but let me go to images and create a, wait, no, this is not the one. I will have to show you properly on responses API. File tool, prediction, presence penalty. By the way, thinking level also, let me show you where you set thinking level. They have verbosity, which is unique to OpenAI, which is how much is the model a talker or not a talker, which can save a ton of tokens in the long run. But what I really want to show you is, there you go, this is the reasoning effort. So OpenAI requires you to set a reasoning effort, and these are the options, which are very different from what Gemini uses for thinking. And let me also show you, you know, the... Images 1. How will you send it? Check the chat. A lot of folks are mentioning on the right panel. It's on the right panel? Okay. Oh, there. Right. Sure. So here I'm using response API but you

know completion API supports very similar you can see type text input very similar you know here and very similar output here as well however there are always subtle differences that you have to watch out for this is why I like open router or something like that for example they have image URL in like is as the you know type and then this is the shape of the image URL versus if you look at Then this is input image and still image URL is sent like this. Or you could do base input 64 as well. This is the example of that. Once again, type input image and this is the base input 64 image. So you see subtle differences are there when you get to advanced teachers. Why are they using same SDK? Why not have separate? They have their own SDKs, but... The benefit of integrating with existing SDK is that you can the pitch you make to enterprises is because enterprises like I'm a product manager I don't want to allocate people like on like everybody's creating new deals they're giving new discount does that mean are it engineering people just keep integrating that doesn't make any sense. The proposition then becomes don't change any code. You don't have to change a single thing. Just go change the API key and the URL you hit, the API base point. And two variables change and I can claim I'll give you free LLM, I'll give you discounted LLM, I'll give you smarter LLM. So this pitch really works that two variables changes in your environment variables or secrets and everything else just works out of the box. So that's the benefit of OpenAI compatibility. That's why everyone supports it. Otherwise, everybody has to learn your API from scratch. And trust me, some of them are not great API designers. So chat completion only with OpenAI and other LLMs are internally calling or using OpenAI? No, Rajesh. OpenAI created that standard, but everybody has tried to support it in their best way. However, your mileage will vary. They eventually become irritating to maintain. That is why I like some standardization like OpenRouter. We're getting sidetracked into so many items that many of us do not know about. Can we please go back on the original linear paths you started teaching with? Thanks. Sure. I'll go back and see where we can go back. It's a prolonged discussion. If I have to send back, each message responds at some point. It may cross the total number of tokens. Is this practical way? Are we deviating from the topic? Because everyone in the chat is like, are we deviating? We are all right on the topic. We are looking at chat completion API. We just saw that this basic shape is supported. I was showing you the differences, right? So everyone supports chat completion API, but there are subtle differences. So I showed you what is the common parts. And then I showed you what are the different parts. Folks, it is not that we are going to be deviating from the topic. Whatever we are talking in terms of generative AI is for your benefit and what we want or we think is relevant for you as well. Like I said, prompting is not just one topic we will be doing. We are covering multiple topics, so we will cover what we know is necessary for you. What you see in that spreadsheet or in your curriculum is a very... Concise, I would say, curriculum part of it. There's a lot more deep into it. That is what we are showing. And we believe in showing with examples rather than just talking about theory. If you want it to be only theoretical wise, going ahead, we can have a change to it. I am fine with it. It will be much easier for Sid and other mentors also to not actually go into practical stuff and show and just talk theory and then towards the end talk practicality. One thing, option, should we do a quick recap of everything we spoke about? A quick recap of that, will that help folks? Yes? Quick recap? Awesome. I'll start my recap here. This is a chat I was having with AI. Where is the capital of France? It's Paris. Tell me something about it. And it tells me, and we are having a chat. Now, you as a software engineer, as a person, as an AI engineer, You will have to implement this in API. You will have, whether you're building a chatbot or you're building some, you know, advanced automation, you will have to implement this sort of a system. Let's assume you're building simple chat app like ChatGPT. Let's say you're building your own perplexity. You will want to know how to do this behind the scenes, right? Because this is also written by, you know, OpenAI developers. So, I suggest to reduce messaging and listen to stats. Yeah, that also will help. Now, Behind the scenes, what we are doing is, it's like any other software. What does the software do? We send it some inputs and we get the output back. For example, a calculator. I will do 2 plus 2, it will tell me



4. So I send the inputs and I get the output. This is how any program works. So LLM, large language model, is basically like a program, which takes inputs and gives you outputs. In this case, we also know them as Prompts and completions. And another way to look at this is, this is my prompt and this is the response of AI or completion. This is my prompt and completion. And you see this shape happening again and again. The same process when we look at the data that comes out, the chat history, the conversation, this is the shape that emerges. I send a prompt, it sends a response. Prompt, response. Prompt, response. It can go on. This entire shape that we're looking at is called multi-turn chat. It's a chat, which is obvious. This is how we chat. But multi-turn because, you know, like one turn involves two, three, like, responses type. Basically, like, you know, I ask first and it responds. That's the basic turn. Now, you can call this, you know, each individual things that are sent input and output in the chat called messages, which is very natural. We send messages in a chat. And we literally store them as a list of messages. And we use this thing called role to differentiate who sent what. Both are messages, but one sent by user, which is your prompt, and one replied by the assistant, which is the completion. You can call this entire chat a chat, a conversation, a thread. It's an ordered list of messages. It says start here, the next, the next, the next. That's all there is to it. This is... what you know what we did we started with is we somebody is running an LLM let's say maybe i am running another one all right and i want to be able to send it prompts and get completions back right And, of course, like, you know, in UI you can do it, but as a software engineer, your real power is basically being able to build software that can do that for you, right? And now we are starting to look at, you know, when this LLM runs, I can see here it's multi-term chat, and I can see here it's multi-term chat. It's the same thing. But when you do it in API, like, the same thing appears slightly differently. The same data you can represent as XML, as JSON, as, you know, I don't know, some bytecode, etc. The fundamental structure of the data does not change and this is where it boils down to. I have a list of messages which is the entire chat history of that particular chat window and the model I'm talking to. These are the two things that you basically need to know to run any inference, any input output or any conversation with the LLM. And this exact shape, in fact, captured in what we are calling ChatCompletions API. These are the common bits. Like, literally, if you send these two, practically, you can get a response from any AI, even open source ones. ChatCompletion is one of the oldest interfaces built by OpenAI and adopted by everyone. And all of them will support ChatCompletion API. Do they have their own APIs, their own preferred way of doing things? Yes. For example, OpenAI has something called response API, Anthropic has messages API, et cetera, et cetera. All right? They have their own preferred way of doing things. But OpenAI, like chat completion API, is the universal standard. It's also called OpenAI standard. It's also called OpenAI SDK standard or OpenAI SDK compatible. All right? And I showed you quickly that obviously OpenAI supports it, right? I can go back here. Yeah, there you go. I think image input, there you go. So I can see the model and the messages. In Python, this will look like this. In Python, literally calling from the SDK chat completion API, sending the model, and sending the list of messages. This basic thing is supported. Same thing for Anthropic. They call it OpenAI compatibility. And if I go here, right, literally how you do this, from OpenAI, import OpenAI, this is TypeScript, let me switch to Python. There you go. Same thing, initialize it, and then send this. All right. And remember, I'm changing the base URL of the API. So I'm sending this to Anthropic servers now, even though I'm using OpenAI SDK. But I'm still sending, this will not go to OpenAI. Maybe some telemetry data might go if they are put in their SDK, but... The final data will only go to Anthropic. And I used my Anthropic API key. Same thing on Google as well, which is very rare. They literally support this very thing. And same thing goes here. I set my Gemini API key. I change the base URL so the data is actually sent to Google APIs. I set my model. I send my messages history and I'm talking all right so all three and you can go to any you know um lab etc there's open router as well open router is an aggregator all right they connect with every other area so you don't have to and they also inherently like their basic stuff is chat

completion API I can go to the API reference I can go to generations they call it slightly differently but uh you know we send literally the same thing okay this is not the one Point providers, OAuth, Anthropic messages, response. Let's look at overview. All right. One second, folks. There you go. Completion request format. This is chat completion. And it's in the quick start guide. If I go here, you can see if I change this to Python, where can I change this to Python? All right. I don't know why it's not allowing me to change to Python. But you can see the shape is still the same. Model, messages. I'll be back in one second, folks. So sorry. All right, back. So they also support, you know, like OpenAI SDK, et cetera, and chat completion API. So everyone supports chat completion API. And this is the standard way to send a prompt and get a response. And this is what you will use when you build your own chat GPT, when you build your own AI agents, et cetera. Now, I hope that is much better and clearer and, you know, without distractions. And this is what we have spoken about so far. All right, now I will take some questions. In multi-term chat, could you please explain how tokens are measured? We'll come to tokens and this thing in post-session. Today will be too early. Can we correlate this out to the OutSkill platform? Where are we in the platform? I'm not sure what you mean to say. How to make sure I use my API tokens carefully and not exhaust my credit or balance? Rate limiting is the best pattern. Introduction, which one is commonly used? Chat completion API or open router? OpenRouter is an aggregator. So they are actually somewhere in between right here. They don't make their own model, but they integrate with everybody. And then they give you their own API, which is basically chat completion API. So OpenRouter itself supports chat completion. They are not one or the other. The problem OpenRouter is trying to solve is you don't have to generate 10 keys and keep them. You generate one key and manage it in one place. So ideally, we can call any model from any SDK. You can call any model from OpenAI SDK. The same is not true for Anthropic or Gemini SDK. Only for OpenAI SDK because they are the ones who created this standard. But if you can hack around, sure, you can call other labs from somebody else's SDK. So this is essentially prompting various LLMs to Python code, correct, or any other API. Yes. And how to call and prompt LLMs. Yes. Not on their GUIs, but in our code. Precisely. Thank you for articulating that. What is your recommended way? OpenAI or router? Folks, it depends. My initial instinct is always to go with open router because often I want to test different models and then it becomes annoying. So at least I would want to check out Sonnet's response on that, GPT-5 response on that. So I usually tend to go for open router. You might find like, you know, other favorites. And there are some certain cases in which case you have to integrate with the main lab directly. I'll talk about those use cases and those reasons in the last session on cost. Are all of them not required to conform to a standard format? This is the standard format that they conform to. Everything else, there is no legally binding requirement. It's just if you're a good player, if you care about it or not. But these required parameters are supported by everyone as it is. In a prolonged discussion, if I have to send back each message, eventually you will run out of that? Yes. We'll talk about that later on. Which LLM to use for chat completion? Any, how to decide? Start with basic ones that you use every day and then you can try different ones. Should I store my session in a cache to maintain the context? You can and if you're, especially if you use, if you use OpenAI like partners or Anthropic, they might save it for you. Okay, they offer that option. Some people will not save it for you. So I would just say, yeah, it's practical to just save the responses, right? Can we use OpenRouter using Java code? Yes. Sending attachment is also done by a chat completion API. Yes, you can do that. Everyone has different way of sending attachments. OpenRouter is basically like a marketplace and aggregator. Yes. OpenRouter and Bedrock are similar, are we right? Yes, they are similar in a sense that, you know, Bedrock will also sit on this layer between them. And Bedrock will also offer you chat completion API, yes, AWS Bedrock. The critical difference is that OpenRouter supports, what they do is they integrate with each of these labs. What Bedrock does importantly is they run it on their own hardware. So, for example, AWS Bedrock is one of the few ones who can run Anthropic Claude on their hardware. But they are not integrating with Anthropic for that. They

have a contract with Anthropic, and at the same price as Anthropic's API pricing, they can sell you this inference as well. So, slightly different, but in the end, yes, practically the same effect. But you will not get as many models on AWS Bedrock as on OpenRouter. And then open routers, bedrock, are similar. I've covered that. Are there other solutions? I didn't understand the role of open routers. Stick to API schema. Right now, open router is confusing. Not an issue, folks. You can ignore open routers and only focus on whichever one is your favorite AI model. Let's assume OpenAI. Let's assume OpenAI, right? How does multi-app use different models? Can you articulate a question a little more, Sneha? Which open SDK is best to start for enterprise companies? RAG and vector DBs? Are they used for chat completion? We will talk about the topic you're asking as we move forward. Today, we are covering fundamental basics. Let's first learn how can you talk to a model using API, using code, right? Because you're going to write code. You're not going to write, you know, the user UI. So let's focus on this first and we'll add more complexity as we go on. For companies where they want to keep data in their own vicinity, they can use Bedrock and HostVidya. So I'll not get into that. Which one is recommended for production? If you mean like from these labs, any one which works for you. Most popular are OpenAI, Anthropic and Gemini, Google. Those are the most popular one. Why go for OpenAI SDK for each model instead of their native SDK? So folks, is the same problem imagine which one you would you prefer would you prefer one integration okay you write one integration and it gives you 300 plus models this is option number one or you do about 10 plus integrations and it gives you access to about 50 plus models which one would you prefer option a or option b which one do you prefer depends on the cost sure all right now this One integration to get everything is exactly what the chat completion API is. This is the common agreed upon API, not officially agreed into any standards. It's just that this is the first API which was usable that was built by OpenAI. So everyone after them adopted it. The core piece of this entire chat completion API is a list of messages and a single model. There are other options, those are not as standardized. But these two are standardized and these two are what you require to do any sort of chatting. And there are a lot of tools, people have built a lot of libraries, a lot of helpful things for you to bridge the gap. You don't have to worry about it. If you want a single unified API over it, it's possible. There are multiple flavors available out there. SDK helpers, etc. Or you can write your own using AI agent, right? But this is the standard that you're looking for. This is the common standard among everyone now. All right? And this is 10 plus integrations is this scenario. You integrate with OpenAI, then integrate with Anthropic, then new NVIDIA launches, then you integrate with NVIDIA, then you realize, oh, DeepSeq is, you know, running ahead. So I have to integrate with DeepSeq as well. And these integrations require writing provider-specific code. There may be benefit of it. You might require it. We don't know. But the great starting point everywhere is ChatCompletions API. Very simple. I hope that works. Different between LLM and RAG, Ratnakar, I would suggest for time being, ignore the word RAG. We have not spoken about RAG. I have not mentioned RAG. We're not talking about RAG today. When they will come for RAG, we'll talk about RAG. Today, we'll focus on LLMs. Any reason they call chat completion, actually the chat will be called again and again for more chat histories, yes. Is there a reason? I'm not the biggest fan of prompt and completion and this is where the chat completion comes into the picture. This completion done chat in a chat format is where chat completion comes from. It is just, you know, pre-historical concepts. Back in the day, we were truly calling them prompts. So, it stuck around and that's where it's from. If you prefer a different API, Unfortunately, that's not the one available. There are some options you can look at of rephrasing it. Do I need to have all the keys to be loaded in OpenRouter? No. You don't have to do that. You get one key and you get access to almost any model. But will they cover the model's specific capabilities in common layer as more and more features start coming in? You will have to read the documentation. Like there's no guarantee they will cover. And usually what they do is they cover it in chat completion. Like there are all kinds of options. They might not cover it. They might cover it like, you know, but not very great support, maybe not very reliable.

So as you get more model specific, let's say I only want to use Claude for vision, then I might be better off integrating for that use case with Anthropic directly, right? Because I might get easier integration, et cetera. So there are reasons to go there as well. But again, I'm telling you, this is a good starting point. When you don't know what you have to specifically do this can be your starting point and as and when you require you can do specific integrations right to benefit from those particular models right so I hope that makes sense any reason they call chat competition I've covered that do I need to have all the keys I've covered that okay uh I would advise only ask questions regarding topic yes as your openness Yeah, if you want any questions that you feel, Sid, are not related to the topic or basic, I can take it in the chat. I've been doing that. Basically, regarding OpenRouter, I will keep taking in the chat. It's fine. Thanks. Is it good analogy for compare Ollama and AWS Bedrock as local LLM host? It's a decent comparison, sure. I will use interface-driven development to work with multiple providers with their specific providers. We'll try to make it plus play. Sure, Mohit. That's a good experiment. In OpenRouter, do we need to configure which? No, you can though. Also, can we get multiple input lines from multiple users in parallel? We share the user session. You want to have group chat with LLM? Yes, it's possible. If you want to see an existing implementation of it, you can see a lot of Discord, which have multiple users talking to AI. It's slightly tricky and it can be very costly, so don't burn your hands over it. But if you want to see, like, basically what you're saying is group chat. Like how we have on messages on you know whatsapp or telegram I can add a bunch of my friends how about I also add an LLM you can definitely do that um the official api does not support it but nothing is stopping me from changing this into a json of like multiple responses nothing is technically stopping you from adding multiple user messages either So there are multiple ways to accomplish this. My favorite implementation remains Discord channels. There are multiple of them. Do check it out. And they have community-funded proprietary LLMs talking in the chat. And they are used for community experiments, et cetera. So also, can we get multiple input? Yes. All right. And open router internally goes to individual model from different vendor, right? Yes. How to catch errors when we do an LLM call? Which errors? Can we please get into specific question at the end of session? Sure. I'm just making sure that we have covered the basics. Can we use multiple LLMs to handle user prompt and get the combined response? Yes, you can. Can we go to the integration practical? Sure. All right, let's do the integration practical. Let's integrate the basics and see how that looks like. I asked Sona to build it. So let me just generate my API key and I should be up and running. Sona is not somebody I want to talk to. I will probably talk to 4.1 Mini. I have to generate an API key. And API key, most API keys, I think the only exception here is, what do you call them, Gemini. They have some free usage. From AI studio however every other uh you know if you want to open here you have to go add credits there you have to like spend money from your credit card debit card to add money credits there and then you can use their api otherwise it will throw an error that you don't have enough balance right so here I've already added my you know credits etc I'm just generating a temporary api key so I can demonstrate it to you and we can try to use it here right api keys are supposed to be secret folks not share them I've saved this let me try okay failed to execute post message window up okay I cannot be uh like run it here got it let's try to fix it should be able to run it and let me show you the code quickly uh again this would have like used you know um some sort of like completion api with open router it's my default completion api it's doing all of this manually crazy And now I can ask it to use like, you know, can you see it doing this manually, manually setting the headers, etc. So I'm going to stop that one and say, oh, it can't install the SDK. Never mind. It can't install SDK here. Maybe I can build one, you know, in lovable quickly. If possible, it doesn't connect and I can download and show it to you folks, not avoid. And I can also show you the code quickly. So we will take, you know, actual code from OpenAI SDK because I don't want you to feel that you have to remember any of this code. You see this code right here? I'm literally copying it, all right? And I'm going to my code files. You remember example.5 we were looking at? Let's create a new one. And this

would be chart completions. And I've literally pasted the code from what I, like, you know, what I just copied from there, SDK. Let me save this. Okay, this is not... Go here. New file, chart.py. There you go. So once again, from OpenAI, import OpenAI, and all of this jazz, I have evoking chart completion, create. And let's change this. Change this to what is the capital of France. Now, how do I know which model I have to send here? I can go here, and they have a list of models in their documentation. So I'll go to their docs. They have models right here. I'm going to the list of models because I need the exact slug. All right? Exact slug I would need. Let's look at 5.4 mini. Sounds good. And can you see model ID? This is what I copy. This is what goes in there. This button right here, I copy that. I go back here. This is my model. All right? And the next thing we'll have to do is just run inference, but a couple of things. Now, to run this file, you know, this is just a quick revision of what we were talking about earlier. Let me clear this. I'm in my local codebase and, you know, this is underlined. Does anybody know what that could be? We could try and run this and see what happens. A module not found error. Anybody remembers why that happens? Install it. All right. So I have to install. All right. So pip install OpenAI. I'm genuinely pleasantly surprised, folks. Great that you remember. And now let's try it once more. If this works, it'll again fail, but we'll come to that. There you go. And now we get another error. And be patient with errors and read them. OpenAI API error. The API key client option must be set either by passing the API key or setting this as an environment variable. We haven't actually given it my API key. Alright, so what I'm going to do is I'm going to like find a way to give it my API key and usually what you do is create an environment file because you don't want to store it, you don't commit it as a part of a beta repository otherwise it will leak. OpenAI API key and I will generate and paste in API key right here. How am I generating API key? I'll quickly show you but I can't show you the key but I can show you how I generate it. I click here and go to API dashboard and I'll have to log in again. My 2FA comes to bite me back. If folks have sharp eyes, you notice that issue as well. Yes, it'll be in the ignore list. Keeping them secret is one of the most important things that you have to do for your security. That's literally the heart and soul of it, actually. 811. Also, folks, by chance, if you push your API key directly on the code and you push it on GitHub, do you know a lot of times OpenRouter disables that API key? It will always disable it. It will always disable it. And I love that feature. It's actually good because no one can misuse that API. I went here to API keys right here and I can create a new secret key and I can name it whatever I want. Let's name it March 23 and I will select the project and I can restrict it, etc. I'll just create a basic one. My bad, I should not be showing it. Yeah, I can't show you the actual key because it's literally like, you know, password, etc. But hey, I've shown you the steps to generate. And I will just save it and be back. All right. In my .env file now, I have saved the OpenAI API key. It roughly looks like this. OpenAI API key equal to whatever is the key. All right. This is what it looks like. And we will now ask one of our agents. I think I should be able to do that code myself. But let's load. What was it? Load environment. Python. This is called python.env. I don't remember any of the package names, so I just chat GPT a lot, and this is their simple syntax. So we will do pip install first, and then I will literally just copy-paste this code, and that should be it. And let's run it again. API client must be, okay, it's not happy at all. So I am going to import the API key directly perhaps. I'm just gonna see like where do they set that parameter in their Python SDK so I don't make a mistake there otherwise it'll be very annoying. All right, here we go to Python. I have to set it here as an API key. Let me quickly ask strategy instead of like banging my head against it. In Python, how do I pass it the API key? I have to, oh, I have to do load.env. You're right, thanks. I completely forgot about that. Let's try that. looks like it might work and there you go can you see here folks my bad where did i go okay crazy one second did i click somewhere bad terminals yeah there you go can you see here folks can you see this part i ran this and chat completion object was done the content is paris and we asked what is the capital of france and there are a whole bunch of other options given but that is the Core content. To make it more believable, let me ask it, you know, extended

question. Can you write 10 bullet points about it? Now, this response should contain those 10 bullet points. Let me clear this up and let's run the Python file again. There you go. The content is the capital of France is Paris. And here are the 10 bullet points. There you go. We are actually starting to use chat completion API standard, just like that. And I did not invent any code. The only part I knew was how to load this file, which, again, you could have chat GPT, et cetera, to figure that out. This is not the biggest deal. And the entire code here is copy-pasted from their SDK. And it is standard. When you start typing it, you're out of the computer. And we are talking to GPT 5.4. Using the same SDK, I should be able to talk to Gemini as well. I should be practically able to talk to anyone. All right? And print response choices text. How is context memory set here is the entire history of chat that you send. How past messages are getting appended, we have to store and append them. Your code will have to do that. How is it responding about Paris? Great question. I'll cover that in one shoot, but towards the end. We all might not have an API key. How to get one? For now, what you can do is, you can go to some place, this place called AI Studio by Google, and they will allow you to generate an API key, which has some free usage. If I'm using only one model, do I still need open router? No, you don't need open router then. So this is AI Studio. You can, do you see here, get API key? You can go there and get yourself an API key. It has some free limited usage. All right? And that is how that works. And I was showing you this thing that even Google supports it, right? So I can go to Gemini API docs. Let's go here. Let's quickly show you that the same setup will work for Gemini as well. And here I will go to, again, OpenAI compatibility, right there. I have to set my API key like this now, all right? And change the base URL to this. Let me write that code as well. I will probably create a new file. I will copy paste the entire thing once again so we don't reinvent from scratch. Here what I want to do is you know first generated Gemini API key and load that and then I have to pass this right here same thing as this I'm literally copying this and placing that that's it here my API key needs to come so I will delete that and write variable API key or this would be I'm not sure I think it would be Gemini So now this is pointing to Google APIs, not OpenAI anymore. And I have to import OS as well. And this GPT-5.4 will not work with them. So they will need something like this. So I'll literally copy that. With that much work, this should work already. And I have to generate an API key. My bad, folks. Just let me quickly generate one. I'll show you if I go here. And these are all my generated one. Right here, there's a, you know, API, create an API key. You fill this form and you will get your API key. And yes, I can't show that here. All right. And I will just copy that. Let's send it out. I'm going to environment file and just literally pasting Gemini API key. And done. All right, I'm back. I said just Gemini API key and now let's test it. Let's run it. It should work because the promise was chat completion works with anything, right? So let's run Python, chat, Gemini. Let's run it. All right. We got a massive response. Let's see what it is. Right here, you can see Quentin is the capital of France's Paris, blah, blah, blah. We got the same response. And then we have... There is something called, you know, extra content, which is thought signature. So they don't actually send you the thoughts, though you can ask for it on their API. They send you encrypted summary of it, and this is the encryption thing, like, you know, in case it's useful. So, but the main content is right here from content, right? And if you wanted a cleaner version of it, let me change the code to print that. You can do whatever. You can store it in DB, et cetera. Let's print. Let's print. exactly the same thing but also get the content out of it there you go and now let me run this code again and this time let's change the question um which ai are you which company in you right this is gemini i hope it can remember like uh you know uh which model it is usually lms don't remember very well i'll run this again okay i am a large language model trained by google there you go so now with the same sdk With barely any change in code, all I changed was just to set up an API key and I literally did not have to change a single thing. My entire setup just works. Same case is with any other lab. This is the benefit of having a common unified API. And this is why you don't have to remember any of this code. Like you have to understand what's happening behind the scenes. We are just literally evoking an

inference by sending it chat history. All right. Now, if I use local LLM, how does it work? no difference uh literally what you will change is this and the api key these two parameters is what continues to change and then you should be able to run your local uh model as well let me see if i can pull one up i was trying to launch lm studio but it did not get launched i don't know what's up with it yeah balaji thanks for answering that basically docs all right import os done can role field be anything no there are only set role uh that are accepted What is a thought signature? It's an encrypted version of the thoughts. Can you explain about thought signature? So Gemini models think. Let me show you an AI studio. They think. They are thinking models. And I can show you the flash preview one. This is flash latest light. Let's go to flash. Gemini 3 flash preview. There you go. So if I ask it, the same thing I was asking over there in the code. Let me copy that. Which AI model are you, etc, etc. And let's see what it answers. And I run it. It directly answers because I have not switched on the thinking part. Thinking level is high. Very strange. What are most important aspects of AI model training? This should make it think. There you go. These are thoughts. The model thinks to itself. And here we can see the UI. And see the content. But by default in their APIs, they will return encrypted content. You can request for unencrypted content as well. This is the thought part and this is the answer part. Similarly, what we have gotten here is the thought signature, which is literally like a hash over the entire thought frame. But this is the main content we are looking at, the main answer. All right, so we spoke with Gemini, we spoke with GPT models. If all API keys cost, then how much will we be able to test practical? We will talk about Hugging Face. You will get a lot of free AI. Don't worry. Can you explain about ThoughtSignature already covered? Do you mean Google, ChatGPT has their own LLM along with different models? Yes. In case your books are not clear, let me draw that out. I thought that might have been clear, but looks like not, which is fine. But I don't want to add to the confusion. Let me quickly clarify a couple of things. We were taking these names, OpenAI, Anthropic. And, you know, let's take XAI and ZAI. These are what we call companies or labs. Okay. Google is also one. These are companies or labs. AI labs as companies. They often create a product, right, which can be an API or UI. For example, OpenAI has GPT. Anthropic has Claudebot AI and Claudebot. Google has Gemini. XAI has GROK, ZAI has GLM. This is their products. And finally, what you use in the end is a single specific model. Each model has version, it has like lineage, etc. And these are AI models. ChatGP, for example, has GPT-40, very popular one. They also have currently the latest one, GPT-5.4. These are OpenAI's models. They have Opus 4.6, Onet 4.6, Gemini has Gemini 3.0, Grok has Grok 4.2, etc. They name it very long. GLM has GLM 4.7. These are their models. They are companies, they are products, and they are models. And you can use OpenAI's model using their API. All using ChatGPT. When you're building an application as an engineer, you will use their API. What are you referring to as unified API? The chat completion API, Shikha. Can we cover with attachments? Not at the moment. First, let's cover the basics. Thought responses are coming before the actual result, but what are you getting as a response? Looks like that. So you see thoughts coming in before because they are streaming. I have not written the code for streaming here. Okay, let me again go back to the code and show you what we need. We don't see thought coming in and then word-by-word answer coming in. We see one response because this is run once and it sort of collects all the tokens coming in. Once it gets all the answers, then it will print in one go. If you want it to stream it, then we have to write the code for streaming. The example is given right here, like right here, streaming. Let me go to streaming. All we do here is put stream equal to true. And then we do this for, you know, for loop over it. And this is the kind of response we will get. All right. For example, the role is assistant and the content, this is how the content is coming up. So let me just quickly copy paste that and show you how that looks like in case you are interested to see that. All I'm adding is stream equal to true. Let me go here. I will copy my file. And, you know, make a copy of it so we can, we don't have to rewrite the code. I'm taking chat and calling that into chat stream. All right, and now I'll go here and I will paste my, you know, copy-pasted code and see where we can put it. So this has to go right here, the

chat completion API, right? And that is done. And instead of this, we can do this as well, but we do for chunking completions and we'll run a for loop over it. I think I can comment this out. And now we can run this and this should stream, you know, token by token, like we are used to seeing it, but this is not Gemini, so it may not think, but let's give it a try. I think chat stream. There you go. It's streamed in literally every response. Now, it may look very dense, but it's literally sending... Can you see? The city has a rich history dating back more than and so on. So it's literally sending token by token. You can render this on UI as well, which will look much better than what it looked like here. But this is the reason why you see they're streaming. It's much faster versus waiting for the entire response. Thanks for covering folks. So if I get Azure OpenAI service, I can add all these models and do the same thing depending on which models they offer. Sorry, where is it picking past conversation from? There is no past conversation. This is the entire history. Okay. For example, I can make fake assistant messages and it will just assume that, you know, that was an incident. Let's do a fun thing, okay? There's no other memory apart from this. What I'm going to do is I'm going to write a response, you know, which you're seeing me write it, so it's not by assistant. And I'll send it as if, you know, it is. And let's see if it believes, you know, that thing. There is no capital of France. Now you have literally seen that I wrote this conversation, not the LLM, but any model I send this to. will behave as if that was the chat history because this is its memory this we are sending this back for the inference and it will assume that its history is there let me um i think um we can now uncomment this so we can get access to the final result as well let's see if that works okay that didn't last line did not work let's Let's find out what it was saying. There you go. I'm sorry, that was incorrect. The capital of France is Paris. Now, why would it say, I'm sorry, that was incorrect? Why would it say that? Because that's all the information it has, all right? While passing the messages, if I change any of the messages in older messages, will they impact anything? Yes, it will change the answer later on. What are we doing with this? I'm just showing you how ChatCompletion API works. Whatever you experienced in ChatGPT, right now, we are doing that in code. right because you will have to write this code and understand how this works in and out right can we give anything for role attribute no there are specific ones i'll talk talk about that how to increase the memory in the model so it keeps 365 days of memory prasad there is no easy answer how many rules are there okay let's talk about role now i think that's logical step at least we have i think it looks like we have gone past the basic questions and now we can you know Getting to the specifics of the API. We already spoke about multiple things. Now, people are asking, there is this role thing here, role user, role assistant. How many roles are supported? Largely speaking, folks, there are four to five roles, okay? And here they are. They're very specific ones. First role is role system, okay? And we'll talk about this logical step in prompt engineering. This is the system prompt. In fact, when I showed you this shape, I was lying. We think our message, what is the capital of France, is the first message that goes to AI. That is not true. In fact, there is another hidden message that is always, always sent to the AI to control it, to maneuver it, to make it behave. It's usually not shown to the user. And this thing is called the system prompt. It sits at the top. I'll show you some system prompts. This is what role system is used for. OpenAI and some other labs, they wanted to retain system control for themselves and they wanted to give another hierarchical control to developers. That is where they brought in role developers. But essentially they are used the same way. This is system by the message, by the system creator, by the developers. Then the next role comes in, role user. This is us. Then next role comes in role assistant. This is the AI response. And the last is role tool. What is role tool? Essentially, when role assistant responds, it can do tool calls. What tool calls are, etc., we'll talk about in the future. This is how they search, they do things. And you have to respond the results in the chat. And we do that using role tools. So you don't have to sweat about this. Right now, let's focus on this. There's role system, developer. You can use them interchangeably. There's a role user, which is for user messages, and the role assistant for



AI messages. Every company and model follows the same five role names. OpenAI has, some people do like, you know, developers user assistant tool, some people do system user assistant tool. So the only difference is, is it system or developers? Otherwise, everybody else follows the same. What happens if I give the role that is not defined? Some APIs will throw an error. Models may misbehave. And some may outright reject your invalid API call. If I want to get data from any predefined document or any database, how to derive? We'll come to documents later on. Tool calling, yes. Tool calling documents is not the topic of today. We'll cover them exclusively in depth. However, I want to make sure now, do you feel confident? Do you understand what chat completion API is? Why it unlocks everything for us, whichever model we want to talk to? And the important piece is the pattern and not the exact code. Does this role come for all LLMs? Mostly yes. I don't think Prasad, you can create a role. I don't think that's available. You have to check, like with your own model, fine-tune model, you might be able to, but mostly like, unless the model was trained on it, it's going to be, act strange. Any built-in function to get metadata info of API like role, etc. They have documented this in their documentation. Like if you go here, you know, I can open messages and I have class developer and class chat completion. They have given a whole bunch of stuff. I will go to Python because it's much easier to understand. I don't know why this is. They made it super dense, their entire thing, but let me show you. So role is developer here. And this role is system. This role is user. This is assistant. And finally, tool call. This one is called role tool. And this one is called role function. Okay, role function is another one, but it is just like almost the same thing as tool, by the way. This is not supported by a lot of folks. I think this is their backward compatibility. Function, my bad. Literally the same thing. And they documented this way. If you go to Anthropic's API, they have given their documentation. I can go to messages, create a message. Here they have given role as well. If I open messages, there you go. Alternating user and assistant. And yeah, that's about it. User and assistant is what they are supporting currently. They cover tools separately. Here is the example. User assistant, user assistant. And see, if you want to include system prompt, you can't set the top-level system parameter. There is no system role. So they don't provide it as a role in their main API. They provide it as a system prompt. I'll show you what that means. This is their own API. This is not chat completion. I'll show you. You see the system? So in Anthropic, this is how you send the system prompt, as a separate parameter, not as a part of messages. But in chat completion, they support that. So yeah, these are the roles that are available and you know, it's all documented in their documentation. You don't have to like, and if you want a unified API order, there are a bunch of options. Again, SDKs, etc. are available. How do I know the right use of role? Meaning if I use an inappropriate role for the task, what happens? Most likely you will get into an error. Explain again the role and its differences. There are two people talking here or there's an AI and user talking here. Let's say there is you. And there is ChatGPT. If I send you a list of messages, do you need to know which message is from you and which message is from ChatGPT? Yes? No? Yes? No? Do you need to know at message level who sent this message? Yes? So the you part is user. The AI part, ChatGPT part is assistant. So these are the main two ones, user and assistant. Everyone clear about that? Everyone clear about that folks? And now we will introduce, we will talk about this in prompt engineering, a special role which is used to guide the overall behavior of the AI model. And that is role system or system prompt or developer prompt. Why do we write it? How does it work? I'll come in a second. But this is for overall guiding of the AI. For example, right here, it's hidden from you, but it's added in the background. And tool you don't have to worry about right now. So you understand role user is you, assistant is AI, and system is the developer or their guidelines. For example, let's say I'm building a chatbot using GPT or Claude for Zomato. So I don't want it to identify as Claude. I want it to say I'm an AI assistant for Zomato. Where will I put that instruction? Where will I put that instruction? I can't edit user message. That doesn't make sense. So I have to insert that instruction somewhere. This is where that instruction goes. What are the key drawbacks of building chat completion API

compared to lower-level LLM in-house models? Covered that toward the end. Additional topics. How can you take an end-to-end example by yourself? It's applied engineering and the answer question is extremely confusing, which I'm not sure if I understand. Not able to follow through at all. Ruchi, let me know what is the last thing that you understood. All right? And long story short, folks, I will, by the way, share this code with you. What I really wanted to do was copy, paste, and write it in front of you. do the AI chat inference we literally were talking to AI and i wanted to show you that there's not like it's not about learning this you know pattern etc it's about knowing what is chat completion api how does it allow you to access any model practically and how do you talk to it how do you send the chat history back to have chat conversation with this information now you are equipped very well equipped now in fact to be able to build this entire interface Except maybe the images. We did not talk about images. But rest of the interface, you can build it yourself. This is ChatCompletions API. And you should be able to build it on top of any model now. Would you please elaborate more on system role? Yes. We will talk about prompt engineering. Then I will elaborate on system prompt heavily. Because as software engineers, you need to know about system prompt. And in real-case workload, which will have longer discussion, do I need to always provide back the accumulated response? Can I accumulate only the response and send it? No, you need to have the entire history or some sort of compactification. For example, I don't know how many of you stream? How many of you use ClaudeCode? How many of you use ClaudeCode? Yes, no? In the chat? Me? Yes? You know when it runs out of context and it's doing compacting? It's compacting, etc.? Yes? Right? And so what happens is, folks, you know, as, and I think this is where we can, like, close first part and take a quick break. The part here is, you know, imagine, what you're asking is long-running chats. What they would look like, let's quickly draw. What we are saying is this chat goes on and on and on. Why am I not able to face it here? Let me take this. All right. I have this. Then I have another message. I have another message. Somewhere, there's a limit. There's a limit to how many messages you can keep in the entire thing. What is this window called? What is the name of this limit? What is the name of this limit? You cannot add more things. It's not session. Context window. Correct. It is the context window. And it's counted in number of tokens. I talked about like token and tokenization towards the end. But this is the context window. It has limited range. You can practically say it can only have so many words. It can only have such length. So when your Claude code hits this limit, around just before 30% above it, it will trigger automatically, Claude code I'm talking about as an example, compactification. they call it differently summarizing compacting same thing all right some people do it behind the scenes automatically and then what it will do it will start a new chat okay and it will start a new thread with now you know somebody added on top so it has some idea about what happened we can imagine this code as like a you know system prompt somebody and this is where we're keeping it so now all this compactified thing goes in the summary And you start a new chat. But it's not the same thing as having real data, right? So that is why you will see performance degrade as well. So this thing is called context window. And largely speaking, we have models ranging from 32K tokens, you know, for open source local model that you can run. 200K is very, very common. And now 1 million context window is becoming more and more common. These are the rough ranges that you will see this in. All right? Compaction, how to set up max size of tokens, step-by-step works. Can compaction be programmed? Absolutely, it can be programmed. For example, you could trigger it at every end tokens, you could trigger it after every turn. It's up to you how you want to use that leverage. It is possible to program it. We have learned how to integrate with chat completion API. Nowhere close to building a fully running chatting tool. Lots of features and meaningful items have been missed like how to append using context, additional parameters. Aman, I agree. However, I cannot run ahead of the time, right? Because if I rush through it, then the fundamentals are lost because adding and removing additional features is not that complicated once the fundamentals are solid because it's an addition on top. For example, you know, once you understand chat completion, now showing you, hey, you know, we want to stream or not

stream was easy example. I only had to show you two parts of the code. And that is how the rest of it is, right? I'll give you a walkthrough of that. Happy to do that. However, I also have to make sure, like, if there is so much knowledge to talk about, right, such as the surface, it is very tempting to run across the... Thank you for watching. Almost every surface we are running some sort of multi-turn chat. Chat completion is the universal standard that is there. Four roles are actually available. And right now we have just spoken about a limit of this, you know, chat completion, this entire window. A limit to all that it can intake and output. And that is what we are calling the context window. It is also known as the context limit. 2K is number of words in each context. No, these are in terms of tokens. These are all in tokens. And I will talk about tokens. I can talk about before or after the break, but let me quickly take all the questions. Role is one metadata, key similar. On top metadata, I need role, messages, model, three things only. You can have more metadata, but the standard one is this. Every provider has their own specifications. Can you repeat the context window's significance? There's a limit to how much history an AI model can process. You can't just keep adding more and more to it. That limit is measured in terms of tokens. I'll show you what tokens are. And right now you can just assume they are words, roughly the same thing, but they are tokens. And these are the standard limits that you will find. I'll quickly show you, for example, if I go to OpenAI. And I go to their models page. I'll go to documents and go to models page. These are their latest models. And let's look at, you know, right here, I'll click and I'll go inside. And here you can see 1,050,000 context window. All right. That is the context window limit of GPT 5.4. Now let's look at some other model, some older model. All right. So for more. I think they do not show the context limit right here outside. Never mind. But we can look at all their models and find an old model I want to show you. For example, I think I want to pick up something like Foro Mini Audio. And you can see here, 128,000 context window. This is in terms of tokens. So each model can have different, you know, Context limits, depending on the use case, depending on how the training was done. Let me quickly show you again this very old model. I think people may remember O3. O3 is right here. I don't know where the O3 full is. Let's look at O3 Mini. O3 Mini had 200K context window, right? Claude's now have 1 million context window across. All right, so this is where context limits come into place. They're very important when you're doing actual implementation. All right, I hope is context window clear to you folks. When I use chat completion API versus response API, can you give me example where each can be used? How much is it remembers from the conversation? Whatever you send back, it technically remembers, but is it using that? Can it use that? Is secondary question. When I use chat completion, we'll talk about differences on specific APIs later on. I use for my code builder 4.1 mini how to bump it up with powerful but free models. You will learn that about on Hugging Face Day. Can system prompt role be added to the chat history? No, you can't manually add it. On API you can, otherwise you can't. Please explain context window and token. Yes, I'll come to context window and token. The context window and all related properties can't be by asking API. And we get by asking API and Python code. So Bharat is asking this limit that we are seeing here, right? And all these properties. Can't we get from an API? Absolutely you can. My favorite way to get this currently is OpenRouter. And by the way, this works for free as well. So you can go to their documentation. And they have a beautiful API called the Models API. Go to API reference. And I will go to, you know, Models right here. And list all models and their properties. And this is what they return for a given model. For example, they have the slug, the name, when it was created, some thoughts about pricing, and you can see here context length. This is the actual context length and you can programmatically get it for hundreds of models. There are other providers as well, but they keep the most up to date because they have incentives to do it because they sell all these things. So this is my favorite way of getting all of this information programmatically. Now, the context window and all related properties can't be already mentioned. Based on token use, credit will be expense correct. While using chat completion API, do we have to manually edit messages every time we ask a new question or your code adds it and saves it?

Can we say content is like input size in terms of token to LLMs? Yes. Can you explain how tokens get translated in terms of text conversation? We'll talk about tokenization. Is there a specific field which can tell the developer how many tokens are left in the current context window? No. However, you can very easily calculate it. Can you please complete one module and then take questions? Sure. I'm not opening the next module of tokenization till I'm done with the questions. Alright. Will this thought-setting interesting case of Gemini be counted as part-consumed tokens? No. They'll be fully consumed tokens. Alright. Now, coming to what is the measure of context length? What do we mean by 8,000? Okay. These 32K, 200K, 1 million, they are in terms of tokens. Tokens is the unit. All right. And before we take a break, do you folks quickly want to look at tokenization? Should we look at tokenization? What is token? Sid, I think we should take the break before. Because tokenization is a topic I want them to be a little more fresher to grasp on. All right, cool. Then I think what I will do is quickly take a break. Let's be back in seven minutes. Karthik, can we afford seven minutes? Yeah, I will put a timer for seven minutes. thon code. So, Bharat is asking this limit that we are seeing here, right, and all these properties. Can't we get from an API? Absolutely you can. My favorite way to get this currently is open router. And by the way, this works for free as well. So you can go to their documentation. And they have a beautiful API called the Models API. I go to API reference. And I will go to models right here. and list all models and their properties. And this is what they return for a given model. For example, they have the slug, all right, the name, when it was created, some thoughts about pricing, and you can see here context length. This is the actual context length, and you can programmatically get it for hundreds of models. There are other providers as well, but they keep the most up to date, because they have incentives to do it because they sell all these things, right? So this is my favorite way of getting all of this information programmatically. Now, the context window and all related properties can't be already mentioned. Based on token, use credit will be expense correct. While using chat completion API, do we have to manually edit messages every time we ask a question or your code adds it and saves it? Can we say content is like input size in terms of token to LLMs? Yes. Can you explain how tokens get translated in terms of text conversation? We'll talk about tokenization. Is there a specific field which can tell the developer on how many tokens are left in current context window? No. However, you can very easily calculate it. Can you please complete one module and then take questions? Sure. I'm not opening the next module of tokenization until I'm done with the questions. All right. Will this thoughts and interesting case of Gemini be counted as part-consumed tokens? No, they'll fully consume token. Now, coming to, what is the measure of context length? What do we mean by 8,000? Okay, these 32K, 200K, 1 million, they are in terms of tokens. Tokens is the unit. Alright, and before we take a break, do you folks quickly want to look at tokenization? Should we look at tokenization? Sid, I think so we should take the break before, because tokenization is on a topic, I want them to be a little more fresher to grasp on. All right, cool, then I think what I will do is, stop sharing, I will quickly take a break. Let's be back in seven minutes. Karth, can we afford seven minutes? Yeah, I will put a timer for seven minutes. Thank you, folks.